

Python Cheat Sheet

Data types

Types

Name	Python type	Examples
integer	int	2; -45; 0
floating point	float	1.014; -12.64; 0.0
string	str	'b'; 'banana'; 'banana cake'
boolean	bool	True; False
list	list	[1, 2, 3]; [-1, 'a', True]
tuple	tuple	(1, 2, 3); (-1, 'a', True)
dictionary	dict	{'banana': 3, 'pear': 42, 'alien fruit': 0}

Type functions

list

Function	Example	Explanation
[]	fruits = ["apple", "grape", "kiwi"]	create an list of strings
x.append(y)	[1, 2].append(3) → [1, 2, 3]	add y to end of x
x.pop(y)	[1, 2].pop(1) → 2	remove element at position y from x
x.reverse()	[1, 2].reverse() → [2, 1]	reverse order of x
x.sort()	[1, 2, 0].sort() → [0, 1, 2]	sort x
x.count(y)	[1, 2, 1].count(1) → 2	count occurrences of y in x
y in x	0 in [1, 2] → False	check if y is in x
x + y	[1, 2] + [3, 4] → [1, 2, 3, 4]	concatenate x and y

string

Function	Example	Explanation
""	name = "Rainer Zufall"	create a string
x.startswith(y)	'banana'.startswith('ban') → True	check if x begins with y
x.endswith(y)	'banana'.endswith('ana') → True	check if x ends with y
x.replace(y, z)	'banana'.replace('a', 'o') → 'bonono'	replace every y with z
y in x	'anan' in 'banana' → True	check if y is in x
x.join(y)	''.join(['First', 'Last']) → 'First Last'	combine elements in y with string x
x.lower()	'BaNaNa'.lower() → 'banana'	convert x to lower case
x.upper()	'BaNaNa'.upper() → 'BANANA'	convert x to upper case
x.split(y)	'ba na na'.split(' ') → ['ba', 'na', 'na']	list of substrings in x separated by y
x.strip()	'banana\n'.strip() → 'banana'	remove leading and trailing whitespace
x + y	'ban' + 'anana' → 'banana'	concatenate x and y

dictionary

Function	Example	Explanation
{}	id_2_name = {}	create a new empty dictionary
set	id_2_name['9606'] = 'Homo sapiens'	add a new key value pair to the dict
get	id_2_name['9606'] → 'Homo sapiens'	access dictionary element using the key '9606' to get its value
update	id_2_name['9606'] = 'Homo sapiens Linnaeus, 1758'	overwrite existing value for given key
delete	del id_2_name['9606']	remove the entry with key '9606'

Slicing

Operation	Example	Explanation
x[y]	[1, 2, 3, 4, 5][0] → 1	get single element
x[y:z]	[1, 2, 3, 4, 5][1:3] → [2, 3]	get elements from positions y to z-1
x[y:]	[1, 2, 3, 4, 5][2:] → [3, 4, 5]	get elements from positions y to the end
x[y:-1]	[1, 2, 3, 4, 5][2:-1] → [3, 4]	get elements from positions y to the second last

Operators

Arithmetic operators

Name	Operator	Example	Result
Addition	+	9 + 2	11
Substraction	-	9 - 2	7
Multiplication	*	9 * 2	18
Integer division	//	9 // 2	4
Float division	/	9 / 2	4.5
Exponent/Power	**	9 ** 2	81
Modulus	%	9 % 2	1

(Note: +, -, *, / and ** have equivalents for direct assignment: +=, -=, *=, /= **=, e.g. x += 5 is equivalent to x = x + 5)

Boolean operators

Name	Operator	Example	Result
Equal	==	9 == 2	False
Not equal	!=	9 != 2	True
Larger	>	9 > 2	True
Larger equal	>=	9 >= 2	True
Smaller	<	9 < 2	False
Smaller equal	<=	9 <= 2	False
AND	and	(9 > 2) and (9 < 11)	True
NOT	not	not (9 > 2)	False
OR	or	(9 > 2) or (9 == 2)	True

Control flow (if, for, while)

If statement

```
if 9 > 2:
    print('9 bigger than 2' )
elif 9 < 2:
    print('9 smaller than 2')
else:
    print('9 equals 2')
```

For loop

```
for i in [1, 2, 3, 4]:
    print(i)
```

While loop

```
i = 1
while i <= 4:
    print(i)
    i += 1
```

Functions and modules

general

Function	Example	Explanation
<code>import x</code>	<code>import(math)</code>	import module x for usage
<code>from x import y</code>	<code>from math import sqrt</code>	import only x from module y
<code>print(x)</code>	<code>print('hello world')</code>	show x on screen
<code>help(x)</code>	<code>help(math)</code>	show documentation for x
<code>exit()</code>	<code>exit()</code>	exit python
<code>type(x)</code>	<code>type(myDNA) → string</code>	type of x
<code>len(x)</code>	<code>len([1, 2, 3, 4]) → 4</code>	length of x
<code>max(x, y)</code>	<code>max(1, 2) → 2</code>	the larger of two objects (also: <code>min()</code>)
<code>abs(x)</code>	<code>abs(-1) → 1</code>	absolute value of x
<code>range(x, y)</code>	<code>range(1, 4) → [1, 2, 3]</code>	list over numbers from x to y-1
<code>round(x, y)</code>	<code>round(1.205, 2) → 1.21</code>	round x to y decimals

math

Function	Example	Explanation
<code>math.sqrt(x)</code>	<code>math.sqrt(4) → 2</code>	square root of x
<code>math.ceil(x)</code>	<code>math.ceil(0.9) → 1.0</code>	round x up
<code>math.floor(x)</code>	<code>math.floor(0.9) → 0.0</code>	round x down
<code>math.cos(x)</code>	<code>math.cos(0) → 1.0</code>	cosine of x (analogous: <code>math.tan(x)</code> , <code>math.sin(x)</code>)

sys

Function	Example	Explanation
<code>sys.argv</code>	<code>['my_script.py', 'my_dna.fasta']</code>	parameters with which the script called
<code>sys.path</code>	<code>['/Library/Python/2.7/site-packages']</code>	directories in which python searches for modules

re

Function	Example	Explanation
<code>re.search(exp, x)</code>	<code>re.search('xy', 'banana') → None</code>	check if regex exp occurs in string x
<code>re.search(exp, x).group(0)</code>	<code>re.search('ana', 'banana') → 'ana'</code>	check if exp in x and show pattern found
<code>re.findall(exp, x)</code>	<code>re.findall('ta', 'tautax') → ['ta', 'ta']</code>	find all occurrences of exp in x
<code>[y z]x</code>	<code>'[A a]lex' → 'Alex'; 'alex'</code>	hits substrings starting with y or z, followed by x
<code>xy*</code>	<code>'TA*' → 'T'; 'TA', 'TAA', ..</code>	hits substrings starting with x followed by any number of y's
<code>\w</code>	<code>'\wno' → 'Ano'; 'zno', '9no', '_no', ..</code>	hits any letter or number 0-9 (upper/lower case) and _
<code>\d</code>	<code>'\d015' → '0015'; '2015', ..</code>	hits any number 0-9
<code>\s</code>	<code>'Mr\s' → 'Mr '; 'Mr\n', 'Mr\t'</code>	hits any white space

Reading and writing files

Reading

```
#open file in reading mode
my_file = open('my_dna.fasta', 'r')

#read a single line
a_line = my_file.readline()

#iterate over all lines in a file
for line in my_file:
    ...

#read all lines
all_lines = my_file.readlines()

#close the file
my_file.close()
```

Writing

```
#open file in writing mode
my_file = open('my_dna.fasta', 'w')

#write string x into file
my_file.write(x)

#write all strings in list x into file
my_file.writelines(x)

#close the file
my_file.close()
```